Data Structures (ETCCDS202)

Unit–2 Assignment: Linear Data Structures

Course Details

Name of the School: SOET
Program (Class / Semester / Batch): B.Tech (CSE / AIML)
Course Title: Data Structures
Course Code: ETCCDS202
Unit Title: Linear Data Structures

Student Name: **Surya Rao**
Roll No.: **2501730243**
Section: **A**

1. Introduction

In this assignment, we implemented various linear data structures including Dynamic Arrays, Linked Lists, Stack, and Queue. These structures store elements sequentially and are widely used in real-world applications such as task scheduling, undo operations, and expression evaluation.
We also developed a real-world application — a Balanced Parentheses Checker using Stack.

---

2. Dynamic Array Implementation

A dynamic array is an array that automatically resizes when it becomes full.
Working Principle
Initially, a fixed-size array is created.
When the array becomes full, a new array of double capacity is created.
All elements are copied into the new array.

Amortized Analysis of append()
The append() operation usually takes O(1) time because the element is added at the end.
However, when the array is full:
A new array is created
All elements are copied → takes O(n) time
This resizing does not happen frequently.
👉 Therefore, over many operations, the average time per append is:
Amortized Time Complexity = O(1)

---

3. Linked List Implementation

3.1 Singly Linked List
Each node contains:
Data
Pointer to next node
Operations Implemented
Insert at beginning
Insert at end
Delete by value
Traverse
Advantages
Dynamic size
No shifting required

3.2 Doubly Linked List

Each node contains:
Data
Pointer to next node
Pointer to previous node
Operations Implemented
Insert after a node
Delete at a given position
Advantages
Traversal possible in both directions
Easier deletion compared to singly linked list

---

4. Stack Implementation (LIFO)

Stack follows Last In First Out (LIFO) principle.
Operations
Push → Insert element
Pop → Remove element
Peek → View top element
Implementation
Stack is implemented using a singly linked list where:
Insert and delete happen at the head

Time Complexity
Push → O(1)
Pop → O(1)
Peek → O(1)

---

5. Queue Implementation (FIFO)

Queue follows First In First Out (FIFO) principle.
Operations
Enqueue → Insert element
Dequeue → Remove element
Front → View first element
Implementation
Queue uses:
Head pointer (front)
Tail pointer (rear)

Time Complexity
Enqueue → O(1)
Dequeue → O(1)
Front → O(1)

6. Application: Balanced Parentheses Checker

We implemented a function to check whether an expression has balanced parentheses.
Rules
Every opening bracket must have a matching closing bracket
Correct order must be maintained
Supported brackets: (), {}, []

Algorithm
Traverse the expression
Push opening brackets into stack
For closing brackets:
Check top of stack
If matching → pop
Else → not balanced
If stack is empty at end → balanced

Test Cases

Input	Output
([])	Balanced
([)]	Not Balanced
(((	Not Balanced
""	Balanced

---

7. Conclusion

Dynamic arrays provide efficient memory usage with resizing.
Linked lists allow flexible insertion and deletion.
Stack and Queue are important abstract data types used in many applications.
Real-world problems like parentheses validation can be efficiently solved using stack.

---

8. References

GeeksforGeeks
Python Documentation
Class Notes